Large-Scale and Multi-Structured Databases

My Akiba

Alessio Franchini

Gianmaria Saggini

Cristina Maria Rita Lombardo

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

UNIVERSITÀ DI PISA

CROSSLAB Innovation for industry 4.0

Application Highlights

This application is a platform that helps anime and manga fans to explore and organize a catalog of their favourite series while participating in a vast community.

Main Features

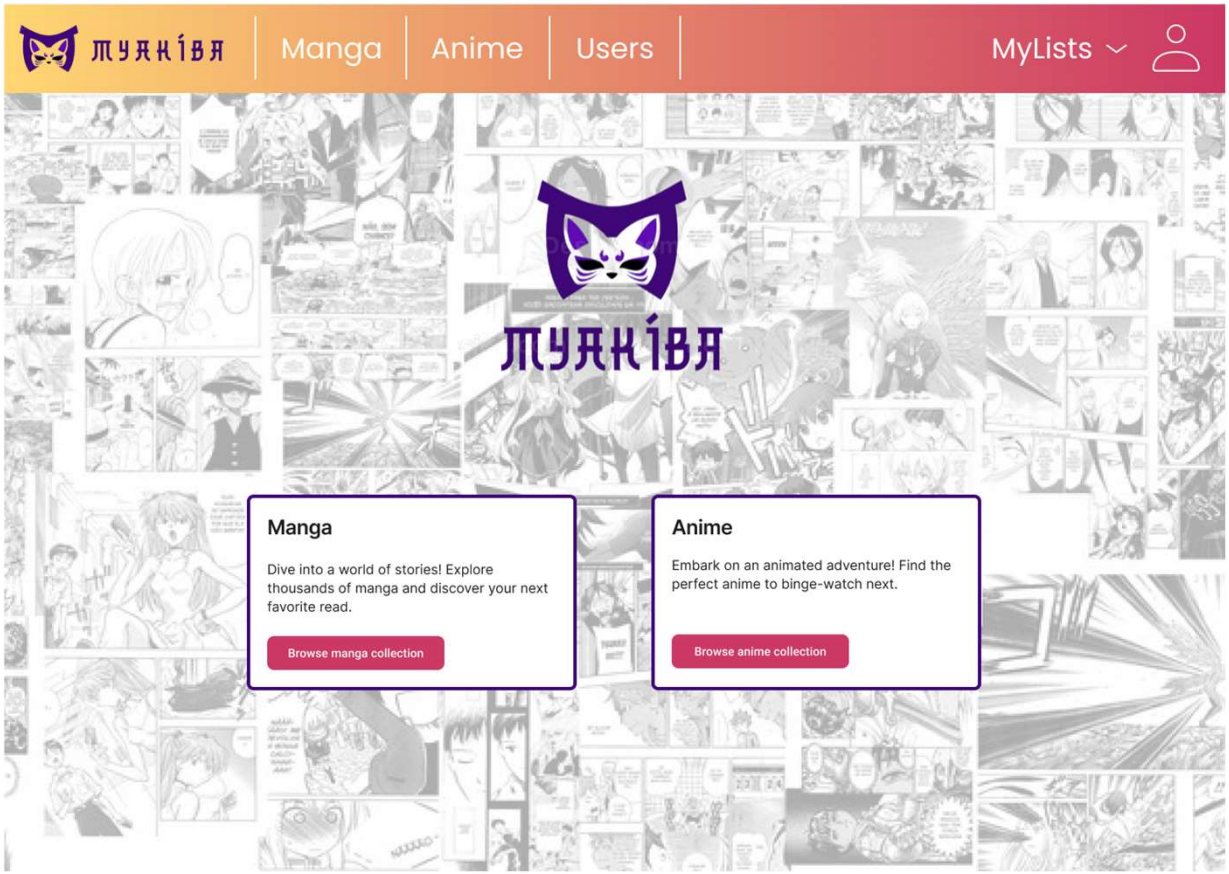
- keep track of your progress by exploiting personal lists such as "Planned", "In Progress", and "Completed"
- search within a complete and updated list of anime and manga and comment on your favorite titles
- connect with others and discover what they are watching

Admin Features

- add or edit entries in the app's global catalog
- update specific details for any anime or manga
- gain insights into app usage and user activity

Actors (i)

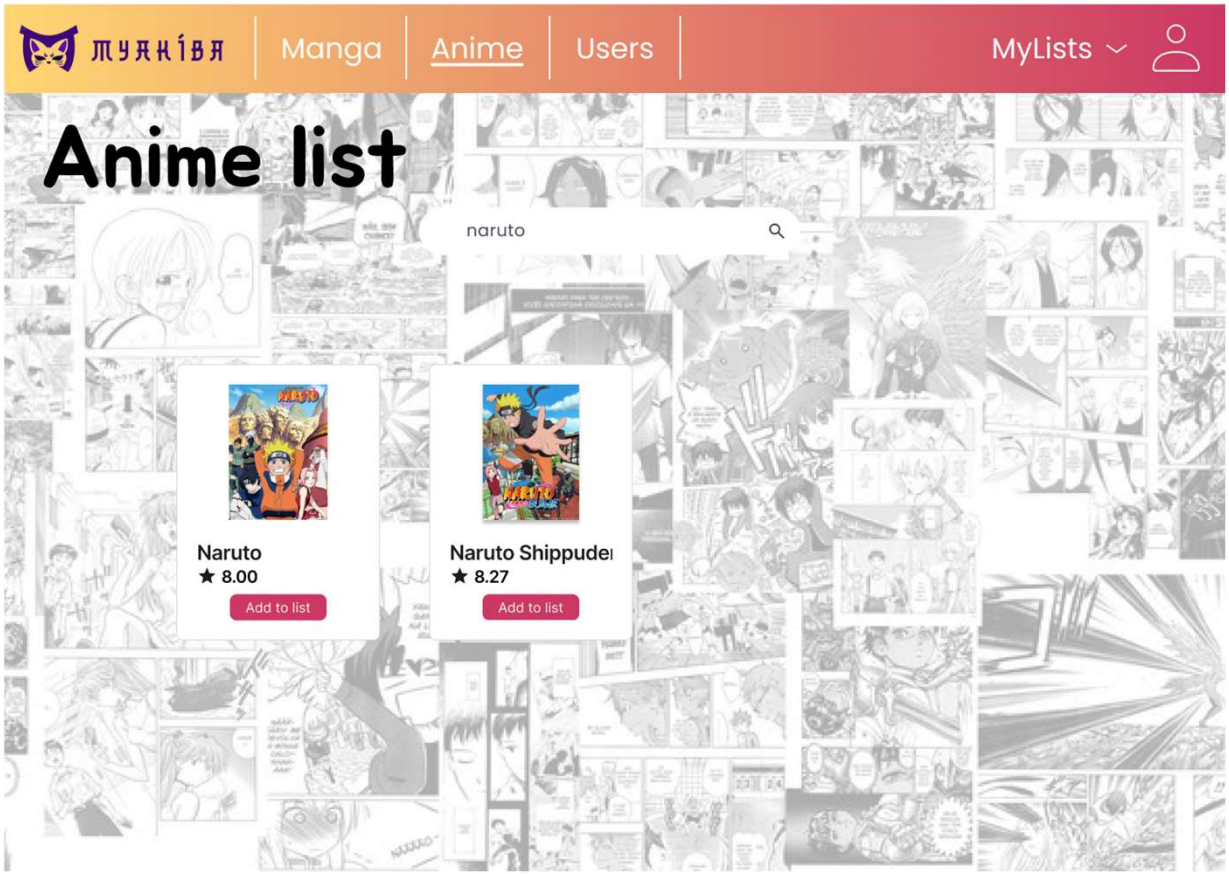
Unregistered user



Main page where the user can choose to browse manga or anime or login to access other functionalities.

Actors (i)

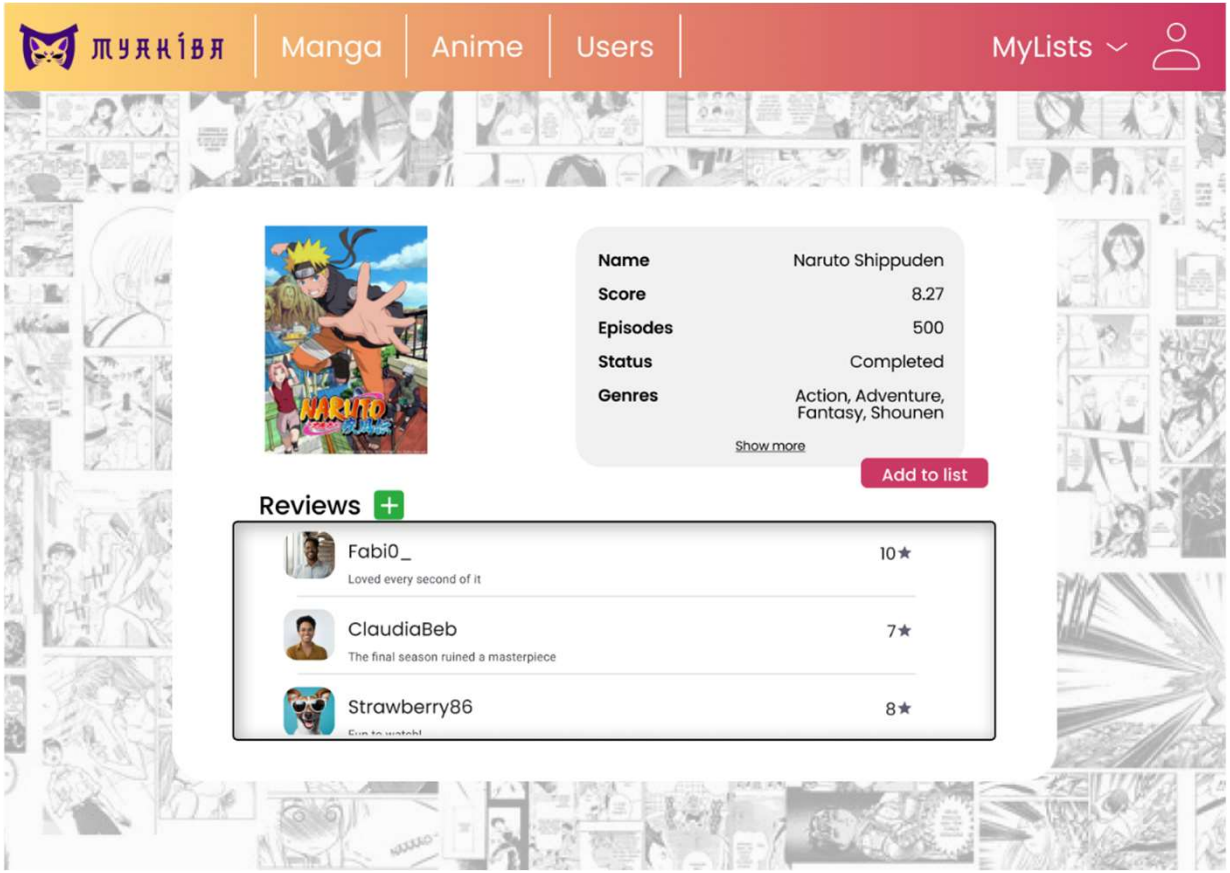
Unregistered user



Users can search for a media by its name.

Actors (i)

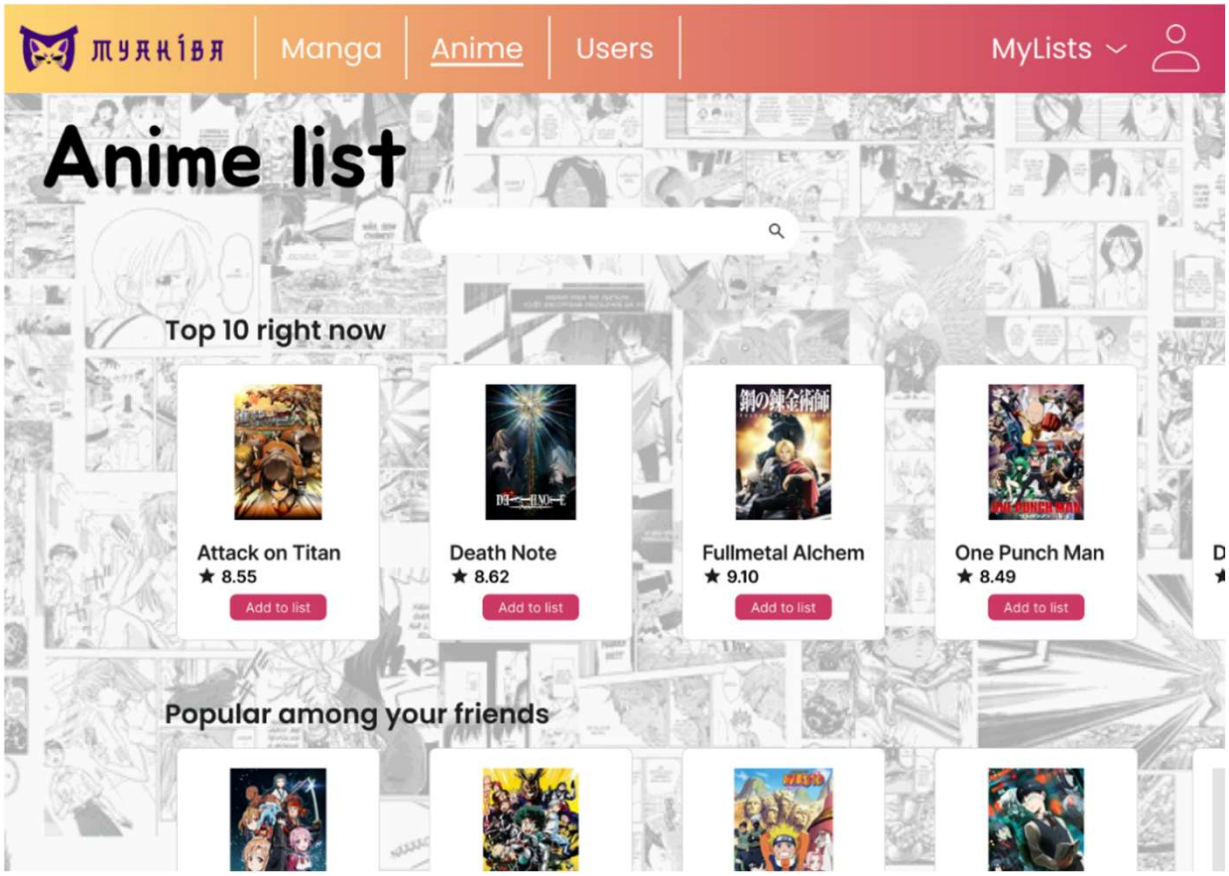
Unregistered user



Each media has its info and reviews displayed in a page.

Actors (ii)

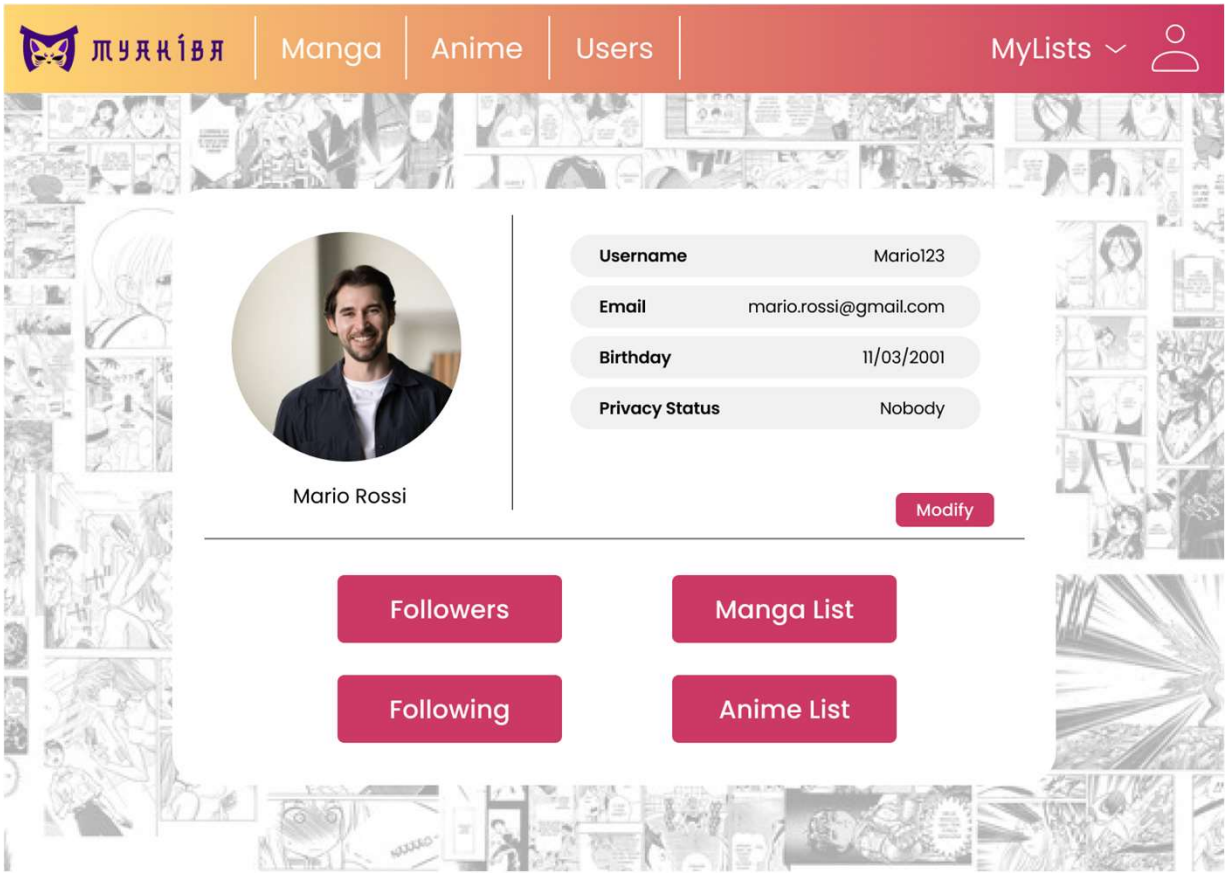
Registered user



Registered users can browse among media recommended for them.

Actors (ii)

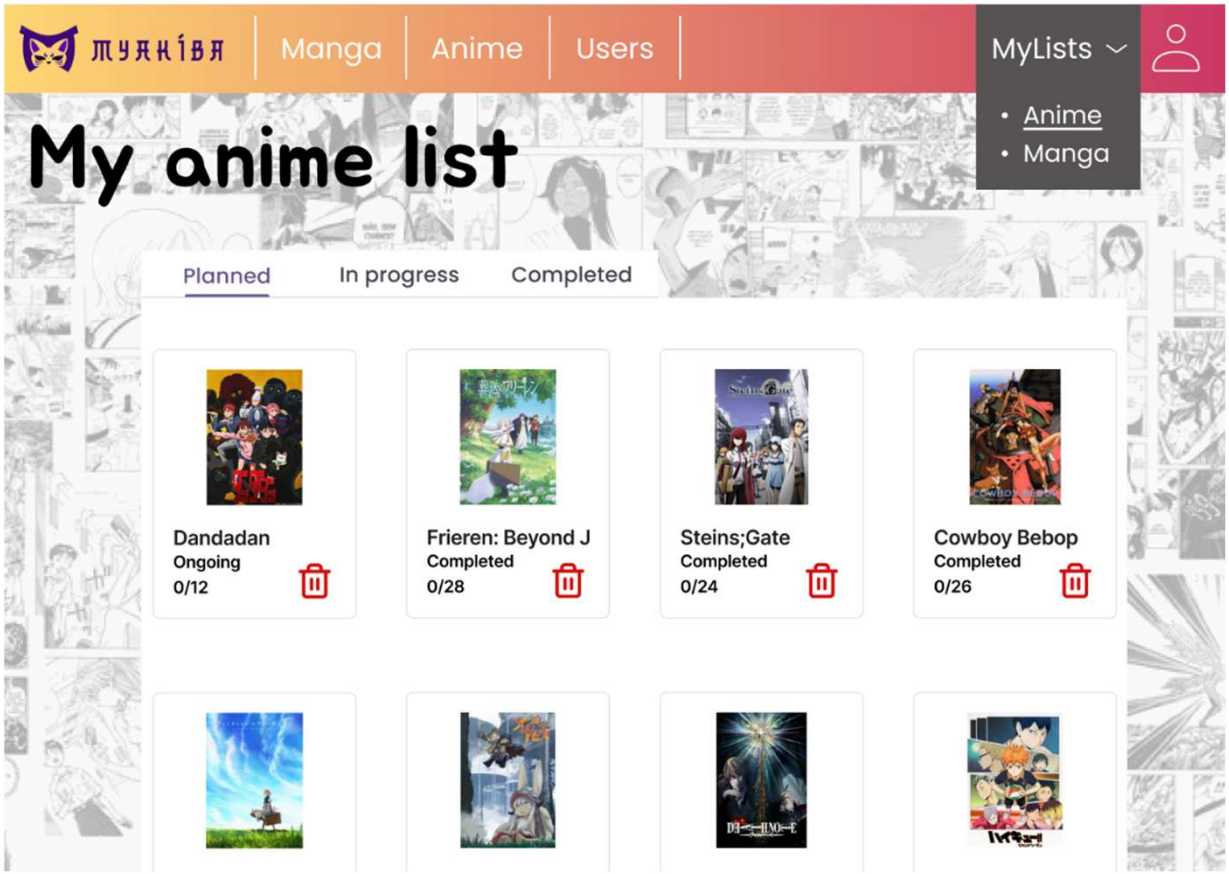
Registered user



Personal user profile.

Actors (ii)

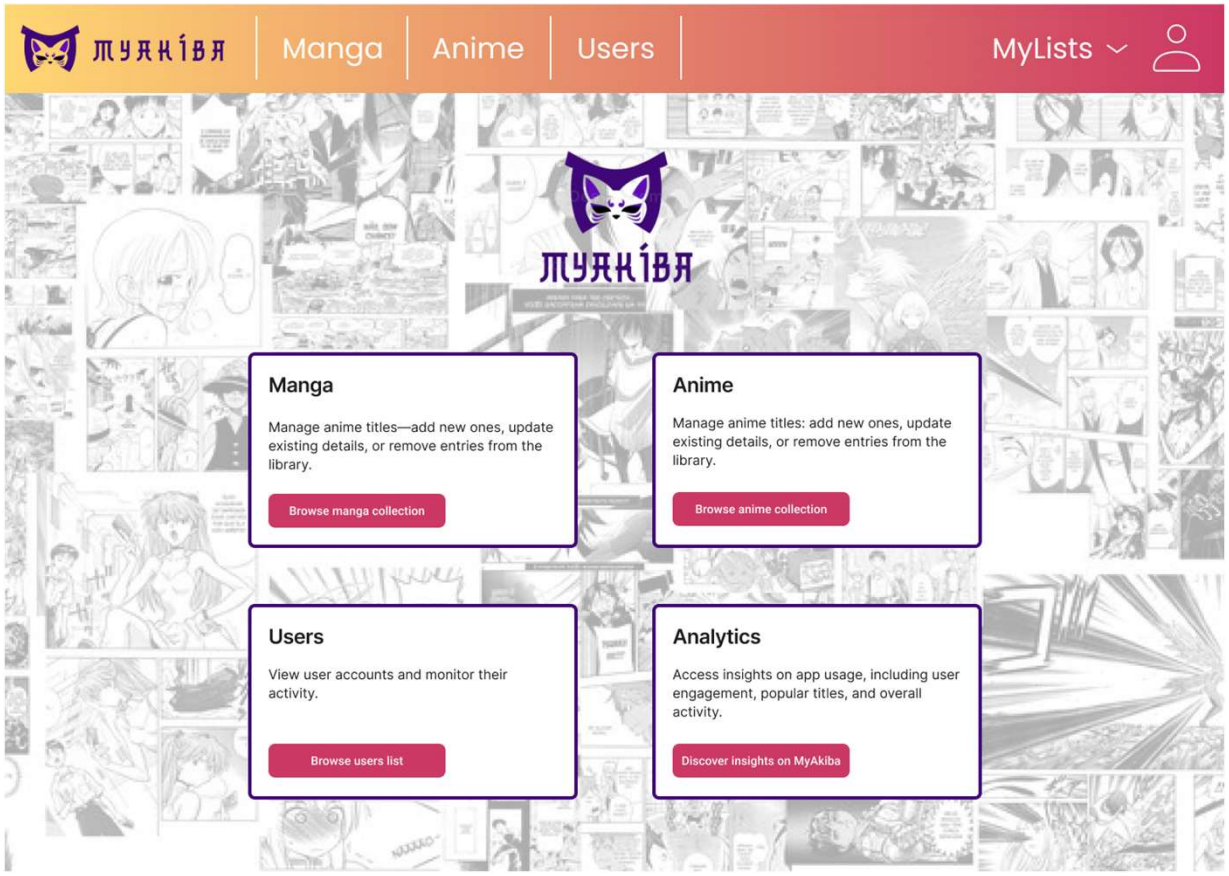
Registered user



Personal user lists, divided by the user progress for each media.

Actors (ii)

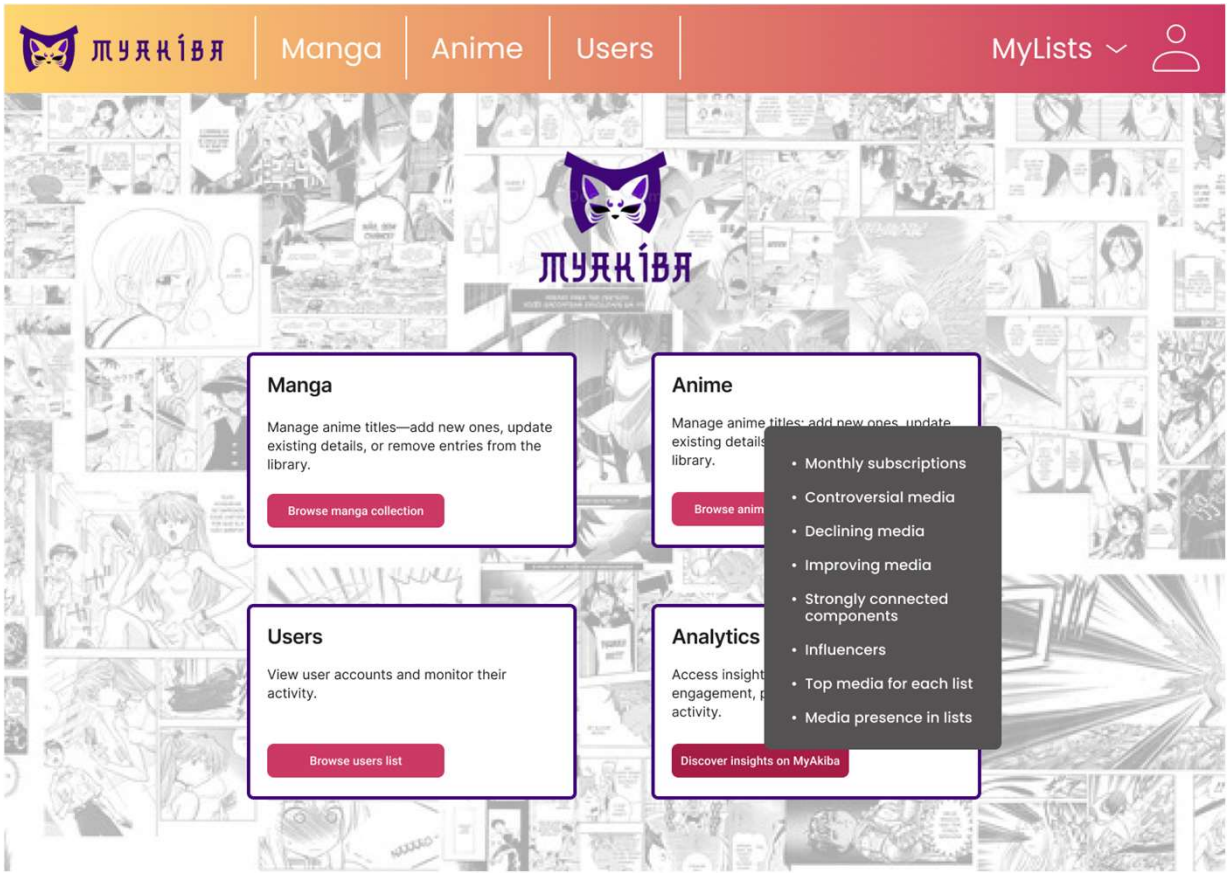
Administrator



Main page with all the functionalities offered to the administrator.

Actors (ii)

Administrator



List of the analytics available.

Dataset Description

Source: MyAnimeList API, Jikan API

Description: Data concerning media details, users' reviews and scores, relations between users and media

Volume: Over 50 MB

Variety: Integrations of data from different APIs

Velocity/Variability: Old data are no longer meaningful, users' lists change continuously and media scores can change in time depending on media releases

Dataset Description

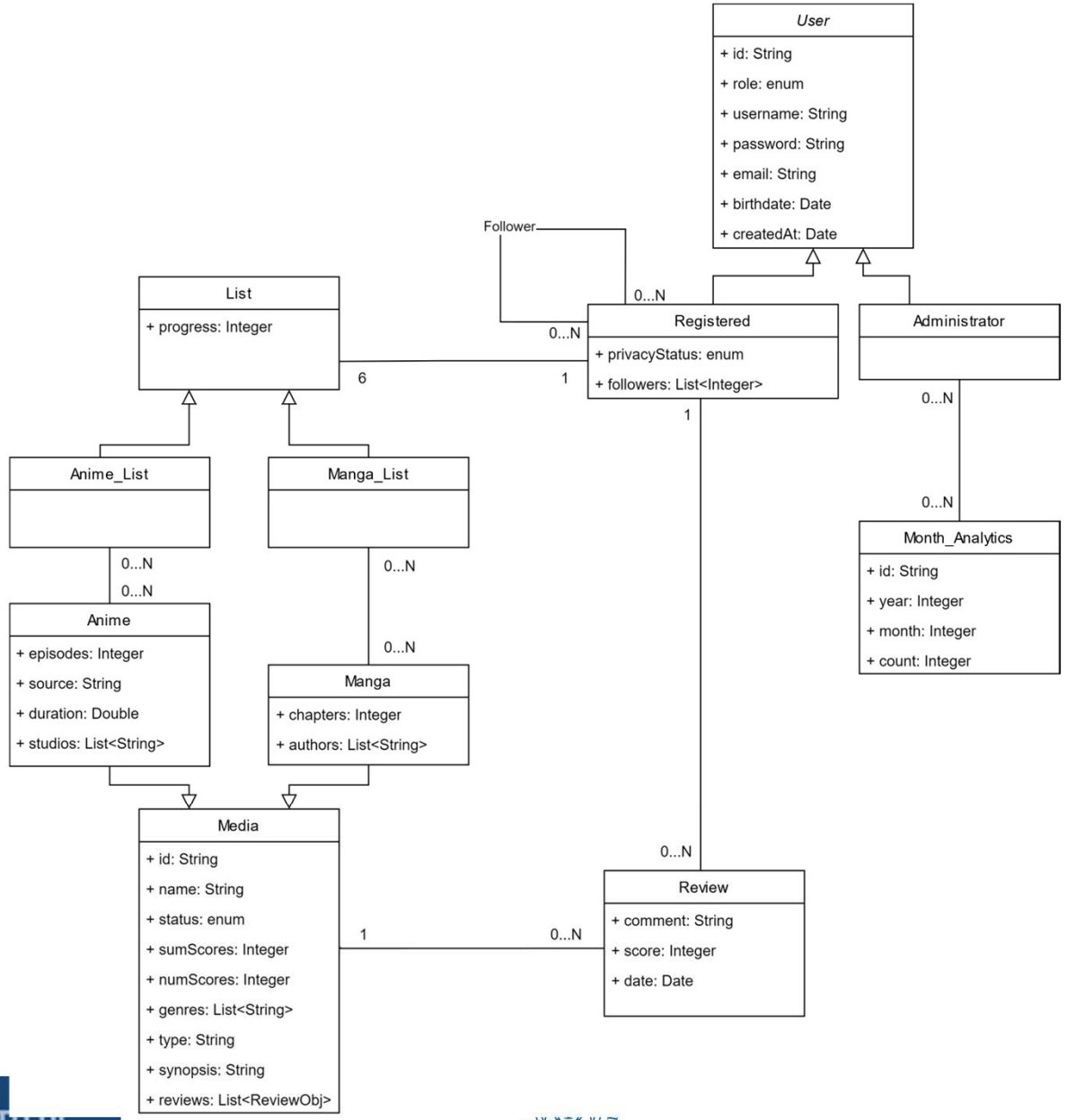
Preprocessing:

- Retrieval from MyAnimeList of users' general information, their lists with the score associated with the media and the media information themselves. We could not retrieve the entire database.

- Retrieval of club information from the website to fetch users with similar tastes generating a more interesting dataset. Then, gathering of usernames from the clubs using the Jikan API, an unofficial and open-source API for the MyAnimeList and assigning them new IDs.

- For both media types, fetching of users' lists details saving them in individual files. Then, retrieval of detailed information for each unique media, including reviews from the Jikan API. The followers' array in the users' information has been generated randomly by sampling a random number of users (between 0 and 50).

UML Design Class Diagram



DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

UNIVERSITÀ DI PISA

CROSSLAB
Innovation for industry 4.0

Application non-functional requirements

Product requirements

- The system must be capable of returning to the user quick responses.
- The system must be highly available. That is, there should not be down time.
- The system should be able to scale to accommodate an increase in user traffic, for instance when a new episode of a popular anime is released, without any degradation of the response time and availability.

Implementation requirements

- The system must be developed using Object-Oriented Programming languages.
- The system must expose its functionalities by means of RESTful APIs, according to OpenAPI Specification.

Security requirements

- The system must use a basic authentication protocol to secure access.
- The system must encrypt users' passwords.

Document DB Design

User collection


```
{
  _id: '6a1c1930-9811-4b00-aa85-64bea06770e8',
  role: 'USER',
  username: 'gianma',
  password: '$2a$12$e2dOgfNSb5T2oIS2R0HDHuKFRdAmIGklUTe260kZcsGe2NOavLLKO',
  email: 'gianmaria.saggini@gmail.com',
  privacyStatus: 'FOLLOWERS',
  birthdate: 2001-03-01T00:00:00.000Z,
  createdAt: 2025-01-08T21:06:53.252Z,
  _class: 'it.unipi.myakiba.model.UserMongo',
  followers: [
    '65397404-1b08-421b-a21e-4d6c54e7ecfb'
  ]
}
```


Manga collection


```
{
  _id: '4742',
  name: 'Papa to Odorou',
  status: 'COMPLETE',
  type: 'tv',
  episodes: 16,
  duration: 390,
  studios: [
    'StudioDeen'
  ],
  source: 'manga',
  genres: [
    'Comedy'
  ],
  synopsis: 'Imagine two irresponsible siblings (
  numScores: 1,
  sumScores: 7,
  reviews: [
    {
      userId: '503',
      username: 'vatu',
      score: 7,
      timestamp: 2022-08-05T07:42:48.000Z
    }
  ]
}
```


Anime collection


```
{
  _id: '124973',
  name: 'Tsuihousha Shokudou e Youkoso!',
  status: 'ONGOING',
  type: 'light_novel',
  chapters: 0,
  authors: [
    ' Gaou',
    'Yuuki Kimikawa'
  ],
  genres: [
    'Fantasy'
  ],
  synopsis: "Backstabbed by those he considered frien
  numScores: 1,
  sumScores: 5,
  reviews: [
    {
      userId: '455',
      username: 'NICKAIL',
      score: 5,
      timestamp: 2017-12-09T07:48:07.000Z
    }
  ]
}
```

Document DB Design

Month with most registrations for each year (analytic)

```
@Aggregation(pipeline = {
        "{ '$match': { 'createdAt': { '$gte': ?0 } } }",
        "{ '$group': { '_id': { 'year': { '$year': '$createdAt' }, 'month': { '$month': '$createdAt' } }, 'count': { '$sum': 1 } } }",
        "{ '$sort': { '_id.year': 1, 'count': -1 } }",
        "{ '$group': { '_id': '$_id.year', 'maxMonth': { '$first': { 'month': '$_id.month', 'count': '$count' } } } }",
        "{ '$project': { '_id': 0, 'year': '$_id', 'month': '$maxMonth.month', 'count': '$maxMonth.count' } }"
})
List<MonthAnalytic> findMaxMonthByYearGreaterThan(Date year);
```

Most controversial media for each genre (analytic)

```
@Aggregation(pipeline = {
        "{ '$addFields': { 'variance': { '$pow': [{ '$stdDevPop': '$reviews.score' }, 2] } } }",
        "{ '$unwind': '$genres' }",
        "{ '$sort': { 'genres': 1, 'variance': -1 } }",
        "{ '$group': { " +
                "    '_id': '$genres', " +
                "    'anime': { '$first': { 'id': '$_id', 'name': '$name', 'variance': '$variance' } } " +
                "} }",
        "{ '$project': { '_id': 0, 'genre': '$_id', 'id': '$anime.id', 'name': '$anime.name' } }"
})
List<ControversialMediaDto> findTopVarianceAnime();
```

Document DB Design

Top declining/improving media (analytic)

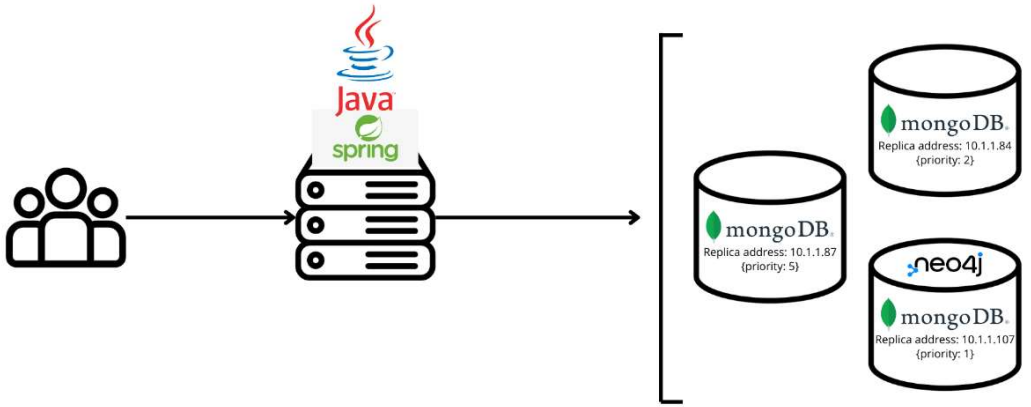
```
@Aggregation(pipeline = {
        "{ '$addFields': { " +
                "    'averageScore': { '$cond': { " +
                "        if: { '$gt': ['$numScores', 0] }, " +
                "        then: { '$divide': ['$sumScores', '$numScores'] }, " +
                "        else: 0 " +
                "    } }, " +
                "    'reviews': { '$slice': [ { '$sortArray': { 'input': '$reviews', 'sortBy': { 'timestamp': -1 } } }, 5 ] } " +
        "} }",
        "{ '$addFields': { 'recentAverageScore': { '$avg': '$reviews.score' } } }",
        "{ '$match': { '$expr': { '$lt': ['$recentAverageScore', '$averageScore'] } } }",
        "{ '$addFields': { 'scoreDifference': { '$subtract': ['$averageScore', '$recentAverageScore'] } } }",
        "{ '$sort': { 'scoreDifference': -1 } }",
        "{ '$limit': 10 }",
        "{ '$project': { '_id': 0, 'id': '$_id', 'name': '$name', 'scoreDifference': 1 } }"
})
List<TrendingMediaDto> findTopDecliningAnime();
```

Top 10 media (recommendation)

```
@Aggregation(pipeline = {
        "{ '$match': { '$expr': { '$or': [ { '$eq': [?0, null] }, { '$in': [?0, '$genres'] } ] } } }",
        "{ '$addFields': { 'averageScore': { '$cond': { if: { '$gt': ['$numScores', 0] }, then: { '$divide': ['$sumScores', '$numScores'] }, else: 0 } } } }",
        "{ '$sort': { 'averageScore': -1 } }",
        "{ '$limit': 10 }",
        "{ '$project': { 'id': 1, 'name': 1, 'averageScore': 1 } }"
})
List<MediaAverageDto> findTop10Anime(String genre);
```

MongoDB Replica Set Configuration

We deployed 3 replicas for MongoDB and 1 replica for Neo4j.
The nodes are configured with different priorities: 5, 2 and 1. This last node is also the node in which we have installed the Neo4j service, minimizing its workload under normal circumstances.



MongoDB Replica Set Configuration

Read preference: nearest

Given the high volume of read requests expected in our system and the requirement of having a quick response to users' requests, this choice helps to balance the database workload, preventing bottlenecks on the primary node.

Write concern: majority

In our application the write operation has a high volume, especially in the context of users lists, and their update. Having only three replicas, this choice is the best trade off, ensuring that most reads (even from secondaries) reflect the most recent data. The slightly increasing write latency shouldn't be an issue considering the limited number of replicas used.

MongoDB Indexes

User collection

- Email: this index is fundamental to speed up the login and the registration.
- Username: this index would help with the registration, but most importantly it would improve the performance of users browsing, which uses a regex on the username.

	Without index	With index
nReturned	1	1
executionTimeMillis	23	13
totalKeysExamined	0	3510
totalDocsExamined	3510	1

- Followers: this index would improve the check on followers when the privacyStatus is on FOLLOWERS. However, it would slow down the follow and unfollow write operation. Since there are 1500 find on a user, of which only about a third will be with privacyStatus=FOLLOWERS, and since there are 4000 follow and unfollow daily, it is not worth adding the index.

MongoDB Indexes

User collection

- CreatedAt: this index would speed up the computation of the "Month with most registrations for each year" aggregation. The aggregation was executed with {'createdAt':{'$gte': ISODate("2020-01-01T00:00:00.000Z")}}.

	Without index	With index
nReturned	60	60
executionTimeMillis	123	10
totalKeysExamined	0	1029
totalDocsExamined	3510	0

Anime and Manga collections

- Name: as the index on the user's name, this index would speed up the browse on anime or manga. Performance tests are similar to the ones on users' name.

Discussion on MongoDB Data Sharding

Shard key: id (for each collection)
Partition algorithm: hashing

This approach ensures that the data is distributed evenly across shards and avoids performance bottlenecks.

We also considered using other fields. However, most searches are based on document IDs, so using another field would have added an extra step to find the sharding key. This would have made queries slower and unnecessarily complicated without providing real benefits.

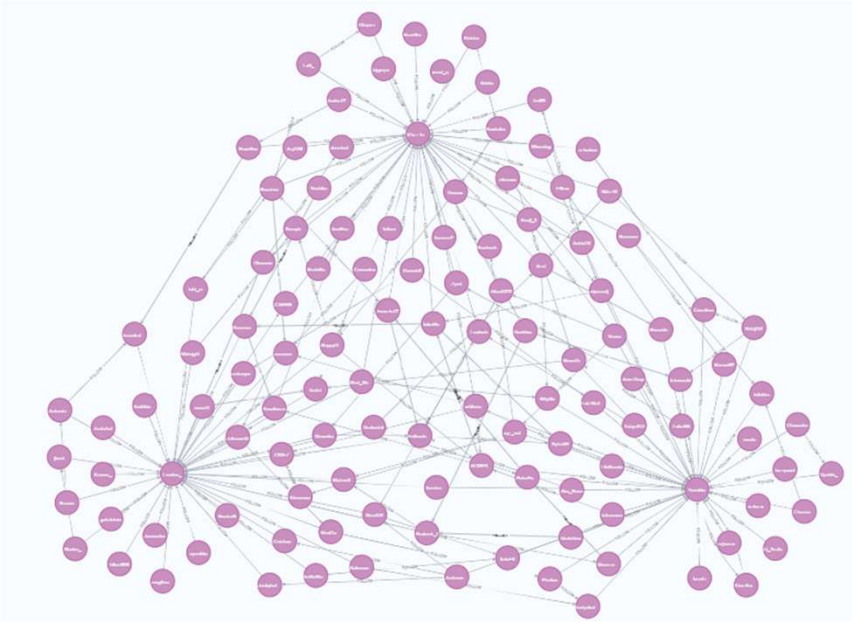
Graph DB Design

Nodes:

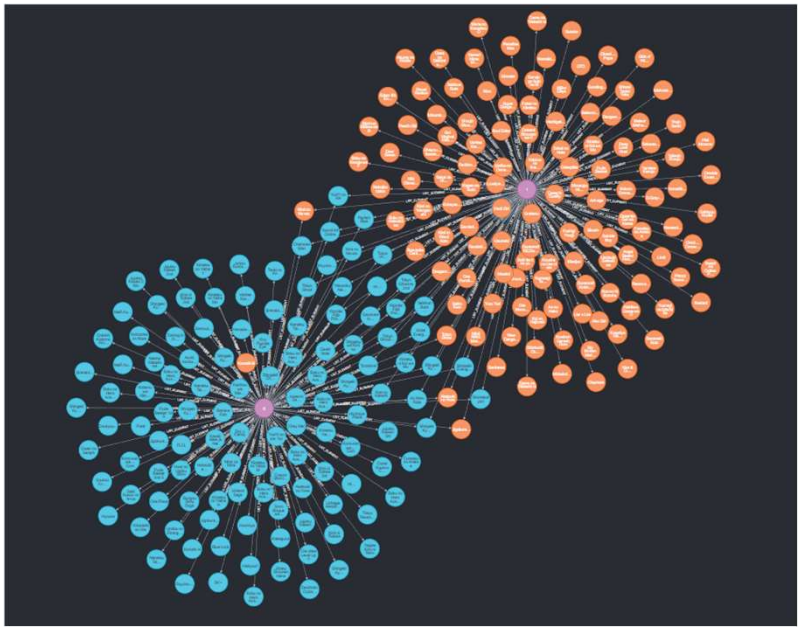
- User {id, username, privacyStatus}
- Anime {id, name, status, episodes}
- Manga {id, name, status, chapters}

Relationships:

- User – FOLLOW → User
- User – LIST_ELEMENT {progress} → Anime/Manga



Sample of users following each other.



Sample of two users and their media lists.

Graph DB Design

Get users with similar tastes

Domain-specific query: given a user, find users that have similar lists. That is, that have a lot of media in common.

Graph-centric query: given a User node, find other User nodes with the highest Jaccard similarity.

```
@Query("""
        MATCH (u:User {id: $userId})-[:LIST_ELEMENT]->(target)<-[:LIST_ELEMENT]-(other:User)
        WITH collect(u) + collect(other) AS sourceNodes, collect(target) AS targetNodes
        CALL gds.graph.project(
          'myGraph',
          {
            User: {     label: 'User'
            },
            Manga: {    label: 'Manga'
            },
            Anime: {    label: 'Anime'
            }
          },
          {
            LIST_ELEMENT: {
              type: 'LIST_ELEMENT'
            }
          }
        )
        YIELD graphName

        CALL gds.nodeSimilarity.stream('myGraph')
        YIELD node1, node2, similarity
        WITH gds.util.asNode(node2) AS user1, similarity
        WHERE gds.util.asNode(node1).id = $userId
        RETURN user1.id AS id, user1.username AS username, similarity
        ORDER BY similarity DESC
        LIMIT 10
        """)
List<UserIdUsernameDto> findUsersWithSimilarTastes(String userId);
```

Graph DB Design

Get popular media among followed users

Domain-specific query: given a user, find most present media in followed users' list.

Graph-centric query: given a User node, find most present Anime or Manga nodes connected to followed User nodes.

```
@Query("""
        MATCH (user:User {id: $userId})-[:FOLLOW]->(f:User)-[:LIST_ELEMENT]->(media)
        WHERE ((media:Anime AND $mediaType = 'ANIME') OR (media:Manga AND $mediaType = 'MANGA'))
              AND f.privacyStatus <> 'NOBODY'
        RETURN media.id AS id, media.name AS name, count(media.id) AS count
        ORDER BY count DESC
        LIMIT 10
        """)
List<MediaIdNameDto> findPopularMediaAmongFollows(String mediaType, String userId);
```

Graph DB Design

Find influencers (graph traversal)

Domain-specific query: find most followed users.

Graph-centric query: calculate the in-degree centrality for each User node, using FOLLOW edges.

```
@Query("""
        MATCH (u:User)<-[:FOLLOW]-(f:User)
        WITH u, count(f) AS followersCount
        ORDER BY followersCount DESC
        LIMIT 20
        RETURN u.id AS userId, u.username AS username, followersCount
        """)
List<InfluencersDto> findMostFollowedUsers();
```

Graph DB Design

Find Strongly Connected Components (graph traversal)

Domain-specific query: find users' community.

Graph-centric query: using FOLLOW edges, apply a community detection algorithm to find well connected User nodes.

```
@Query("""
        MATCH (source:User)-[:FOLLOW]->(target:User)
        WITH collect(source) AS sourceNodes, collect(target) AS targetNodes
        CALL gds.graph.project(
          'graph',
          ['User'],
          {
            FOLLOW: {
              type: 'FOLLOW'
            }
          }
        )
        YIELD graphName

        CALL gds.scc.stream('graph', {})
        YIELD componentId, nodeId
        WITH componentId, collect(gds.util.asNode(nodeId)) AS users
        WHERE size(users) > 1
        RETURN componentId,
               size(users) AS componentSize,
               [user IN users | {id: user.id, username: user.username}] AS userDetails
        ORDER BY componentSize DESC
        """)
List<SCCAnalyticDto> findSCC();
```

Neo4j Indexes

Indexes on id for every type of node.

- Query caches cleared between tests.
- Values are the average of 5 runs.

Query: How many times a media is in each list
Without index: 294ms
With index: 251ms

Query: find similar users
Without index: 428ms
With index: 395ms

Query: find popular media among followed users
Without index: 296ms
With index: 165ms

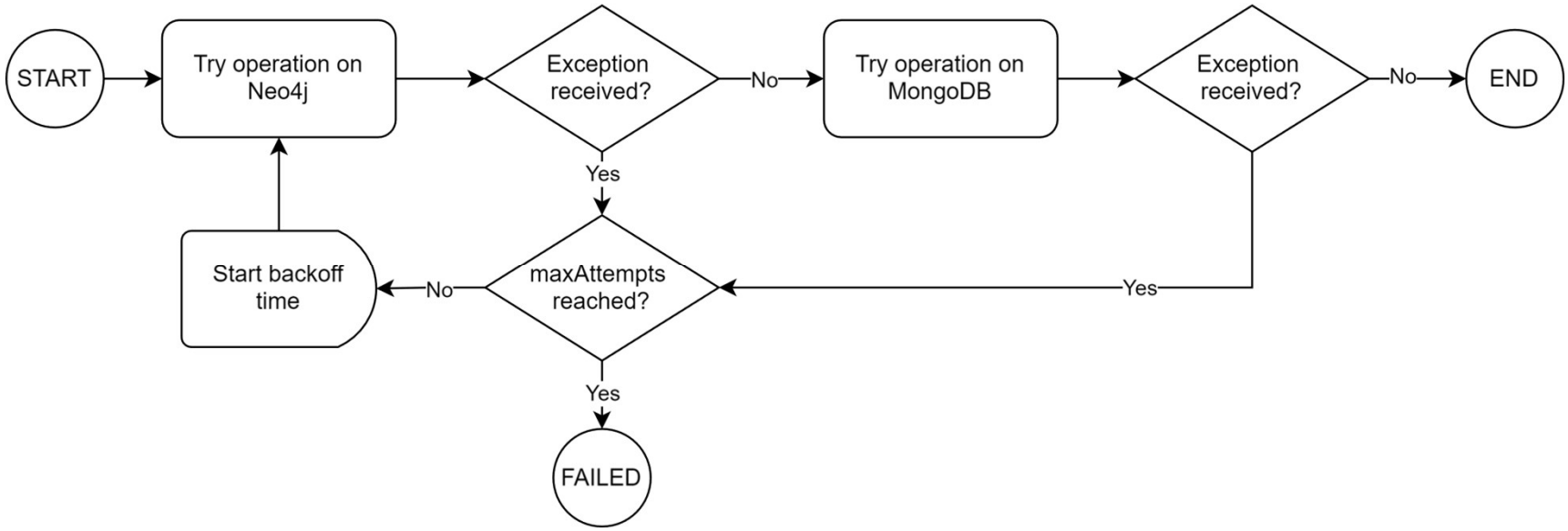
Handling Intra-DB Consistency

Hypothesis: since MongoDB have 3 replicas, we can assume that a fail on connection is almost impossible.

This consistency is managed using retriable spring functions.

- The operation is firstly tried on Neo4j DB, the most likely to fail having only one replica node.
- If the operation succeed, then the operation occurs in MongoDB and if there are no issues the function returns a success response to the user.
- If any of the two phases fails, the operation is retried for three times after a fixed backoff time. If these attempts don't succeed, the operation will fail and return an error response.
- If the operation in MongoDB fails there are no rollbacks in the Neo4j DB, leading to an inconsistency in the database. However, the chances of this case occurring are very slim due to the presence of three nodes handling this database replicas.

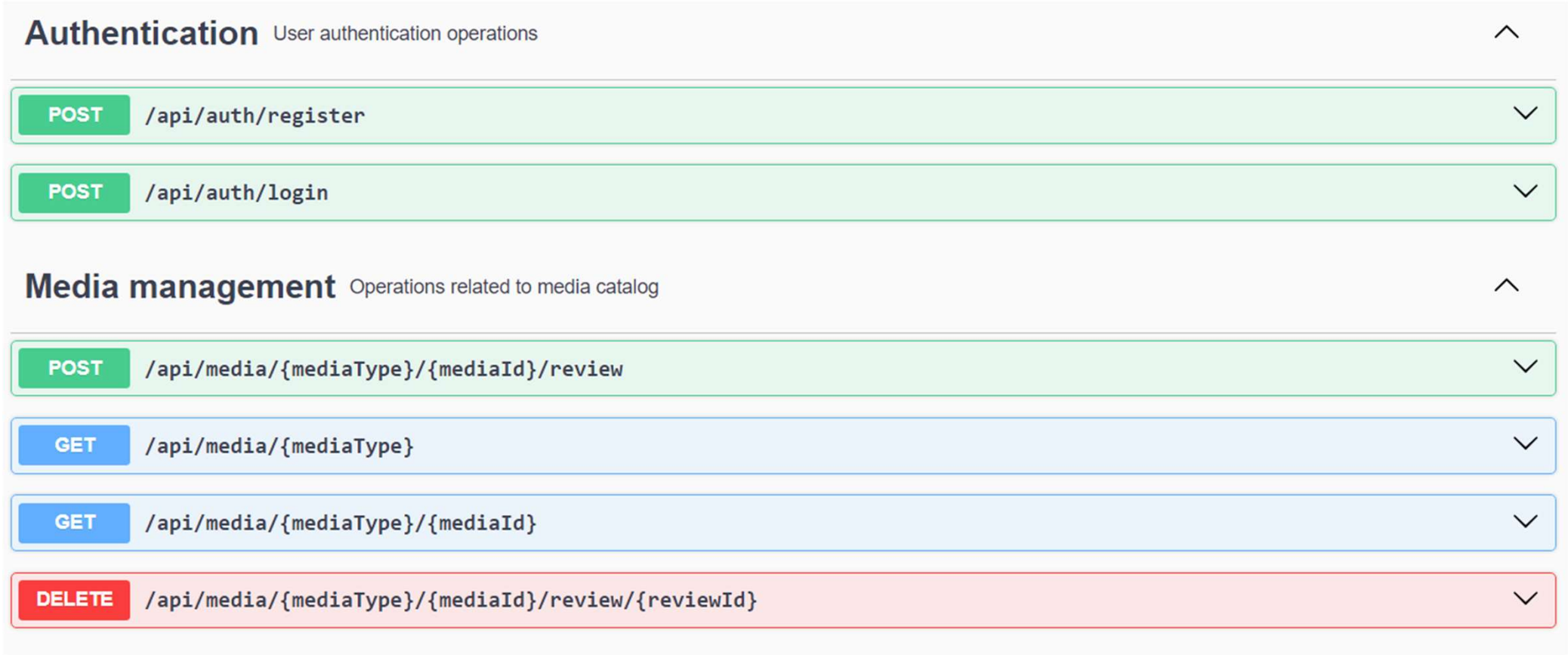
Handling Intra-DB Consistency




```
@Retryable(  1 usage  Gianmaria Saggini +2
        retryFor = {DataAccessException.class, TransactionSystemException.class},
        maxAttempts = 3,
        backoff = @Backoff(delay = 2000)
)
public UserNoPwdDto updateUser(UserMongo user, UserUpdateDto updates) {
```

Swagger UI REST APIs documentation

Swagger doc url: http://localhost:8080/swagger-ui/index.html



Live Demo with Postman